TECHNICAL ARCHITECTURE & OPERATIONS MANUAL

AnimeGAN2 Video Pipeline — Version 4.0

Architecture: Native WSL ext4 Execution • Configuration-Driven Batch Processing • Hardware Target: NVIDIA RTX 3090

---

1. Executive Summary & Architecture Shift

Version 4.0 resolves severe disk I/O bottlenecks identified during long-running 1080p inference runs on Windows-mounted NTFS directories (`/mnt/d/`). Writing 80,000+ sequential JPEG frames across the WSL-to-Windows 9P filesystem bridge throttled pipeline throughput to ~2.3 FPS despite 100% GPU utilization.

The v4.0 architecture isolates storage responsibilities: raw input videos and final rendered outputs remain accessible on Windows NTFS drives, while all intermediate frame extraction, neural inference, frame caching, and video muxing execute natively on Linux `ext4` storage (`/home/admin1/aipipeline`).

Step 3.3 — Deploy Native Inference Engine with Dynamic ETA (inference_1080p.py)

Configure the PyTorch inference script to execute within native `ext4` directories with real-time ETA countdown, processing speed (FPS), and frame caching (`diff < 1.0`).

```
cat << 'EOF' > /home/admin1/aipipeline/inference_1080p.py
import os
import sys
import glob
import time
import cv2
import torch
import numpy as np

PIPELINE_DIR = "/home/admin1/aipipeline"
SUBMODULE_DIR = os.path.join(PIPELINE_DIR, "animegan2-pytorch")

if SUBMODULE_DIR not in sys.path:
    sys.path.insert(0, SUBMODULE_DIR)

print("[*] [v4.0] Loading official Generator from animegan2-pytorch...")
from model import Generator

weight_path = os.path.join(SUBMODULE_DIR, "weights/face_paint_512_v2.pt")
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
net = Generator().to(device)
net.load_state_dict(torch.load(weight_path, map_location=device))
net.eval()

raw_frames_dir = os.path.join(PIPELINE_DIR, "pipeline_output/stage3_raw_frames")
anime_frames_dir = os.path.join(PIPELINE_DIR, "pipeline_output/stage5_anime_frames")
os.makedirs(anime_frames_dir, exist_ok=True)

frame_paths = sorted(glob.glob(os.path.join(raw_frames_dir, "*.jpg")))
total_frames = len(frame_paths)
print(f"[*] Processing {total_frames} frames at 1080p with dynamic ETA tracking
(v4.0)...")

prev_raw_resized = None
prev_out_img = None
cache_hits = 0
start_time = time.time()

for idx, f_path in enumerate(frame_paths):
    img = cv2.imread(f_path)
    if img is None:
        continue

    h, w, _ = img.shape
    new_h = (h // 32) * 32
    new_w = (w // 32) * 32
    img_resized = cv2.resize(img, (new_w, new_h))

    out_name = os.path.basename(f_path)
    out_path = os.path.join(anime_frames_dir, out_name)

    if prev_raw_resized is not None:
        diff = np.mean(np.abs(img_resized.astype(np.float32) -
prev_raw_resized.astype(np.float32)))
        if diff < 1.0:
            cv2.imwrite(out_path, prev_out_img)
            cache_hits += 1
            cur_f = idx + 1
            if cur_f % 100 == 0 or cur_f == total_frames:
                elapsed = time.time() - start_time
                fps = cur_f / elapsed if elapsed > 0 else 0
                eta = (total_frames - cur_f) / fps if fps > 0 else 0
```

Step 3.4 — Deploy Resumable 5-Minute Chunking Batch Runner (batch_pipeline.sh)

Install the state-aware orchestrator that splits source videos into 5-minute segments, tracks progress via `pipeline_state.json`, enables safe interruption/resuming, and stitches master outputs automatically.

```
cat << 'EOF' > /home/admin1/aipipeline/batch_pipeline.sh
#!/bin/bash
set -e

CONFIG_FILE="/home/admin1/aipipeline/pipeline_config.json"

INPUT_DIR=$(python3 -c "import json; print(json.load(open('$CONFIG_FILE'))
['input_dir'])")
OUTPUT_DIR=$(python3 -c "import json; print(json.load(open('$CONFIG_FILE'))
['output_dir'])")
WORKING_DIR=$(python3 -c "import json; print(json.load(open('$CONFIG_FILE'))
['working_dir'])")
STATE_FILE="$WORKING_DIR/pipeline_state.json"

mkdir -p "$OUTPUT_DIR" "$INPUT_DIR" "$WORKING_DIR/segments"
[ ! -f "$STATE_FILE" ] && echo "{}" > "$STATE_FILE"

echo "========================================="
echo "[*] [v4.0 Resumable] Scanning $INPUT_DIR..."
echo "========================================="

shopt -s nullglob
videos=("$INPUT_DIR"/*.mp4 "$INPUT_DIR"/*.mkv "$INPUT_DIR"/*.mov "$INPUT_DIR"/*.avi)
[ ${#videos[@]} -eq 0 ] && echo "[!] No videos found." && exit 0

for video in "${videos[@]}"; do
    filename=$(basename "$video")
    stem="${filename%.*}"
    echo "[*] Processing Video: $filename"

    LOCAL_SRC="$WORKING_DIR/stage1/${stem}.mp4"
    mkdir -p "$(dirname "$LOCAL_SRC")"
    [ ! -f "$LOCAL_SRC" ] && cp "$video" "$LOCAL_SRC"

    SEG_DIR="$WORKING_DIR/segments/$stem"
    mkdir -p "$SEG_DIR colspan"
    if [ -z "$(ls -A "$SEG_DIR"/*.mp4 2>/dev/null)" ]; then
        echo "[*] Splitting video into 5-minute chunks..."
        ffmpeg -y -i "$LOCAL_SRC" -c copy -map 0 -segment_time 300 -f segment -
reset_timestamps 1 "$SEG_DIR/seg_%03d.mp4"
    fi

    segments=("$SEG_DIR"/seg_*.mp4)
    PROCESSED_SEGS_DIR="$WORKING_DIR/processed_segments/$stem"
    mkdir -p "$PROCESSED_SEGS_DIR"
    CONCAT_LIST="$PROCESSED_SEGS_DIR/concat_list.txt"
    > "$CONCAT_LIST"

    for seg in "${segments[@]}"; do
        seg_name=$(basename "$seg")
        seg_stem="${seg_name%.*}"
        final_seg_output="$PROCESSED_SEGS_DIR/${seg_stem}_anime.mp4"

        is_completed=$(python3 -c "import json; d = json.load(open('$STATE_FILE'));
print(d.get('$stem', {}).get('$seg_stem', 'pending'))")
        if [ "$is_completed" == "completed" ] && [ -f "$final_seg_output" ]; then
            echo "[*] [Resume] Segment $seg_name already completed. Skipping..."
            echo "file '$final_seg_output'" >> "$CONCAT_LIST"
            continue
        fi

        echo "[*] Processing Segment: $seg_name"
```

Step 3.5 — Commit & Synchronize Fleet

Commit the v4.0 files to GitHub and run the automated updater on secondary machines.

```
cd /home/admin1/aipipeline
git add -A
git commit -m "Upgrade pipeline to Version 4.0: Config-driven native ext4 batch
processing"
git push origin development
```

4. Fleet Maintenance & New Node Provisioning

Two companion scripts automate deployment across multiple machines without manual configuration.

4.1 Automated Node Updater (update_to_v4.sh)

Run `/home/admin1/aipipeline/update_to_v4.sh` on any existing v3.5 workstation to pull the v4.0 release, generate default configuration files if absent, provision NTFS directories, and apply execution permissions.

```
cat << 'EOF' > /home/admin1/aipipeline/update_to_v4.sh
#!/bin/bash
set -e

PIPELINE_DIR="/home/admin1/aipipeline"
cd "$PIPELINE_DIR"

git fetch origin development
git checkout development
git pull origin development
echo "4.0" > VERSION

CONFIG_FILE="$PIPELINE_DIR/pipeline_config.json"
if [ ! -f "$CONFIG_FILE" ]; then
    cat << 'CONF' > "$CONFIG_FILE"
{
  "version": "4.0",
  "input_dir": "/mnt/d/aipipeline/input_videos",
  "output_dir": "/mnt/d/aipipeline/pipeline_output",
  "working_dir": "/home/admin1/aipipeline"
}
CONF
fi

mkdir -p /mnt/d/aipipeline/input_videos /mnt/d/aipipeline/pipeline_output
"$PIPELINE_DIR/stage1"
chmod +x "$PIPELINE_DIR"/*.sh
echo "[*] Upgrade to v4.0 completed successfully!"
EOF
chmod +x /home/admin1/aipipeline/update_to_v4.sh
```

4.2 Bare-Metal Fresh Installer (fresh_install.sh)

For clean WSL Ubuntu installations, execute `bash /home/admin1/fresh_install.sh` to install system dependencies, clone the repository and submodules, download the `face_paint_512_v2.pt` model weights, configure Miniconda/PyTorch, and initialize `pipeline_config.json`.

```
cat << 'EOF' > /home/admin1/fresh_install.sh
#!/bin/bash
set -e

sudo apt-get update && sudo apt-get install -y git ffmpeg curl build-essential

PIPELINE_DIR="/home/admin1/aipipeline"
mkdir -p "$PIPELINE_DIR" /mnt/d/aipipeline/input_videos /mnt/d/aipipeline/pipeline_output

if [ ! -d "$PIPELINE_DIR/.git" ]; then
    git clone https://github.com/YOUR_USERNAME/aipipeline.git "$PIPELINE_DIR"
fi

cd "$PIPELINE_DIR"
git checkout development
git submodule update --init --recursive

WEIGHTS_DIR="$PIPELINE_DIR/animegan2-pytorch/weights"
mkdir -p "$WEIGHTS_DIR"
WEIGHT_FILE="$WEIGHTS_DIR/face_paint_512_v2.pt"

if [ ! -f "$WEIGHT_FILE" ]; then
    curl -L -o "$WEIGHT_FILE" \
      "https://huggingface.co/akhaliq/AnimeGANv2-
pytorch/resolve/5f2c6e4bf913b2c76de3a842f8a952b41bab9db9/face_paint_512_v2.pt"
fi

if [ ! -d "/home/admin1/miniconda" ]; then
    curl -sLO https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
    bash Miniconda3-latest-Linux-x86_64.sh -b -p /home/admin1/miniconda
    rm Miniconda3-latest-Linux-x86_64.sh
fi

export PATH="/home/admin1/miniconda/bin:$PATH"
if ! conda env list | grep -q "animeenv"; then
    conda create -y -n animeenv python=3.10
fi

/home/admin1/miniconda/envs/animeenv/bin/pip install --upgrade pip
/home/admin1/miniconda/envs/animeenv/bin/pip install torch torchvision --index-url
https://download.pytorch.org/whl/cu118
/home/admin1/miniconda/envs/animeenv/bin/pip install opencv-python numpy

cat << 'CONF' > "$PIPELINE_DIR/pipeline_config.json"
{
  "version": "4.0",
  "input_dir": "/mnt/d/aipipeline/input_videos",
  "output_dir": "/mnt/d/aipipeline/pipeline_output",
  "working_dir": "/home/admin1/aipipeline"
}
CONF

chmod +x "$PIPELINE_DIR"/*.sh
echo "[*] Fresh installation of v4.0 completed!"
EOF
chmod +x /home/admin1/fresh_install.sh
```

5. Standard Operating Procedure (Daily Execution)

> **Operational Workflow:** Drop any number of source videos (`.mp4` , `.mkv` , `.mov` , `.avi`) into `D:\aipipeline\input_videos` , open WSL, and execute `/home/admin1/aipipeline/batch_pipeline.sh` . Completed renders appear automatically in `D:\aipipeline\pipeline_output` named `<original_filename>_anime_v4.mp4` .

Pipeline Stage	Legacy v3.5 Architecture	Upgraded v4.0 Architecture
Source Ingestion	Read directly from Windows NTFS mount	Single bulk copy from `input_dir` to native `ext4`
Frame I/O & Inference	80,000+ sequential writes over WSL 9P bridge	100% native Linux `ext4` read/write operations
File Discovery	Hardcoded single filename (`final_dubbed_video.mp4`)	Dynamic batch scanning (`.mp4` , `.mkv` , `.mov` , `.avi`)
Configuration	Hardcoded paths across multiple scripts	Centralized `pipeline_config.json`